

Inflammation Tracker

Setup & Security Manual

Hostinger deployment

A self-contained Progressive Web App for daily inflammation tracking, synced to your own Koofr account via WebDAV as one CSV file per entry. Hosted as static files on Hostinger; runs offline; installs to phone home screen.

Contents

1	What you have	1
2	Deployment overview	1
3	Prerequisites	2
4	Step 1 — Choose a deployment location	2
5	Step 2 — Upload the files	2
5.1	Method A: hPanel File Manager (browser, no setup)	2
5.2	Method B: SFTP from your laptop (faster for repeat uploads)	3
5.3	Quick check: it loads	3
6	Step 3 — Apply security headers and force HTTPS	3
7	Step 4 — Lock the URL with HTTP Basic auth	4
7.1	Method A: hPanel built-in (easiest)	4
7.2	Method B: Manual <code>.htpasswd</code> (if hPanel option absent)	5
8	Step 5 — Create a Koofr app password	5
9	Step 6 — First configuration in the app	6
10	Step 7 — Set up the Cloudflare Worker proxy	6
11	Step 8 — Install on your phone	8
11.1	Android (Chrome)	8
11.2	Desktop (any Chromium browser)	8
11.3	iOS (Safari)	9
12	Where your data is stored — a quick reference	9
13	Maintenance and recovery	9
13.1	Rotating the Koofr app password	9
13.2	Resetting one device	9
13.3	Losing a device	9
13.4	Backing up the entries folder	9
13.5	Inspecting the data	9
13.6	Updating the app	10
14	Troubleshooting	10

15 Security summary — what protects what	11
Appendix — The .htaccess file (full)	12
16 Your working setup — closing reference	13
16.1 The pieces	13
16.2 What flows where	13
16.3 Where each piece of state lives	14
16.4 Physical disk location of your credentials	14
16.5 How to deploy a change	17
16.6 Sanity-check routines	17
16.7 Backup	17
16.8 If you ever need to rebuild from scratch	17

1 What you have

The application is seven files. They are pure client-side: there is no server-side code, no database, no executable component. Hostinger only serves them as static files.

File	Purpose	Permissions
<code>index.html</code>	App shell, two views (entry, settings)	644
<code>style.css</code>	Mobile-first dark theme	644
<code>app.js</code>	Form logic, queue, view switching	644
<code>sync.js</code>	WebDAV operations against Koofr	644
<code>sw.js</code>	Service worker (network-first, offline fallback)	644
<code>manifest.json</code>	PWA manifest (install metadata)	644
<code>icon.svg</code>	App icon	644

You will additionally create two access-control files during setup:

<code>.htaccess</code>	Apache directives (HTTPS, headers, auth)	644
<code>.htpasswd</code>	Hashed login credentials (kept outside web root)	600

2 Deployment overview

End state once setup is complete:

1. The seven app files live in a folder on your Hostinger site, served over HTTPS.
2. The folder is protected by HTTP Basic auth, so the URL cannot be browsed without credentials.
3. Strong security headers (CSP, HSTS, frame protection) are applied by `.htaccess`.
4. On Android, the app is installed to the home screen and works offline.
5. Entries are saved locally first, then synced to Koofr as individual one-row CSV files written into a folder of your choosing. Each entry has its own file at a unique path, so writes can never overwrite earlier entries.

Note

Storing your own *health data* on your own server is your responsibility. None of the following bypasses ordinary best practice: keep your hosting account 2FA-protected, keep your laptop and phone disk-encrypted with strong unlock credentials, and treat the Koofr app password the same way you treat any other credential.

Note

Adapting to other hosts. Hostinger is used here as a worked example because it has cheap shared hosting with Apache, HTTP Basic auth via hPanel, and `.htaccess` support. Only Steps 1–4 (location, upload, `.htaccess`, Basic auth) are Hostinger-specific. Steps 5–8 (Koofr app password, app settings, Cloudflare Worker, install on phone) are host-neutral. On a different host — Netlify, Cloudflare Pages, GitHub Pages with a custom domain, a small VPS running Apache or Nginx, etc. — substitute that host’s equivalents: its file-upload mechanism, its way of forcing HTTPS, its way of setting response headers (or a CDN in front), and its way of gating the URL with a password (Basic auth, an OAuth proxy, or platform-native access control). The CSP value, the Worker code, and the Koofr side are identical regardless of host.

3 Prerequisites

- Hostinger hosting plan with hPanel access (any plan with PHP/static hosting is fine; the app does not require PHP).
- A domain or subdomain attached to your hosting (the app will live at e.g. `https://yourdomain.example/inflam-XXXX/`).
- A Koofr account.
- A modern browser on your laptop (Chrome, Edge, Firefox, or Safari) and an Android phone with Chrome.

4 Step 1 — Choose a deployment location

You have two reasonable options on Hostinger:

Option A: Subfolder of an existing site

Place the files at `public_html/inflam-XXXX/` where `XXXX` is a random suffix you choose. Accessed as `https://yourdomain.example/inflam-XXXX/`. Simplest; uses your existing SSL certificate.

Option B: Dedicated subdomain

Create a subdomain in hPanel (e.g. `health.yourdomain.example`) and place the files in its document root. Slightly cleaner separation; Hostinger will issue a separate Let’s Encrypt certificate automatically.

Note

Either is fine. The remainder of this manual assumes Option A with the folder name `inflam-XXXX`. Substitute your actual path as needed.

Important

Choose a non-obvious folder name. Avoid `inflam`, `health`, `tracker` as guessable; use something like `inflam-7f3a92` or any random suffix. The HTTP Basic auth in Step 4 is the real lock, but an unguessable URL is a useful second layer.

5 Step 2 — Upload the files

Two ways on Hostinger; pick whichever you find easier.

5.1 Method A: hPanel File Manager (browser, no setup)

1. Log in to Hostinger → **Hosting** → **Manage** on your site → **File Manager**.

2. Navigate into `public_html`.
3. Create a new folder, e.g. `inflam-7f3a92`.
4. Open the folder and use **Upload Files** to upload all seven app files.
5. After upload, right-click each file → **Permissions** and confirm **644**.
6. Right-click the folder → **Permissions** and confirm **755**.

5.2 Method B: SFTP from your laptop (faster for repeat uploads)

In hPanel, go to **Hosting** → **Advanced** → **SSH Access** (or **FTP Accounts**) and note the host, port, username, and password.

From a terminal on macOS:

```
# Copy the seven files (from the project directory)
sftp -P 65002 u12345@yourhost.hostinger.com
# Once connected:
cd public_html
mkdir inflam-7f3a92
cd inflam-7f3a92
put index.html
put style.css
put app.js
put sync.js
put sw.js
put manifest.json
put icon.svg
bye
```

Then set permissions in one shot via SSH (if available on your plan):

```
ssh -p 65002 u12345@yourhost.hostinger.com
cd public_html/inflam-7f3a92
chmod 755 .
chmod 644 *.html *.css *.js *.json *.svg
```

5.3 Quick check: it loads

Open `https://yourdomain.example/inflam-7f3a92/` in a desktop browser. You should see the entry form: four severity buttons (1–4), a location checkbox grid, two notes fields, and a Medication tickbox. If you get a 403 or a directory listing, recheck folder permissions and the presence of `index.html`.

6 Step 3 — Apply security headers and force HTTPS

Hostinger normally enables HTTPS automatically via Let’s Encrypt. We still want an explicit redirect to defeat any accidental `http://` link, and we want a Content Security Policy to harden against script injection.

Create a file named `.htaccess` inside `inflam-7f3a92/` containing:

```
# Force HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]

# Disable directory listing
Options -Indexes
```

```
# Security headers
Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "DENY"
Header always set Referrer-Policy "no-referrer"
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=(), interest-cohort=()"
Header always set Content-Security-Policy "default-src 'none'; script-src 'self'; style-src 'self'; img-src 'self'; manifest-src 'self'; connect-src 'self' https://app.koofr.net; worker-src 'self'; form-action 'none'; frame-ancestors 'none'; base-uri 'none'"

# Correct MIME types (Hostinger usually handles these, but explicit is safe)
AddType application/manifest+json .json
AddType image/svg+xml .svg
AddType text/javascript .js
```

Important

If you later deploy a Cloudflare Worker proxy (Section 7), you must add its origin to the `connect-src` directive, e.g. `connect-src 'self' https://app.koofr.net https://your-worker.workers.dev`. Otherwise the browser will block the sync request even before it leaves the page.

Set permissions: 644 on `.htaccess`.

Verify the headers

From your laptop terminal:

```
curl -I https://yourdomain.example/inflam-7f3a92/
```

You should see `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, etc. in the response. If they are absent, your Hostinger plan may not allow `mod_headers`; raise a support ticket — it is unusual.

7 Step 4 — Lock the URL with HTTP Basic auth

This is the most important security step. Until this is in place, anyone who learns the URL can load the app, read its source, and attempt entries against your Koofr.

Hostinger offers a built-in flow for this. Recommended path:

7.1 Method A: hPanel built-in (easiest)

1. In hPanel: **Advanced** → **Password Protect Directories**.
2. Select `public_html/inflam-7f3a92`.
3. Set a username (e.g. your initials) and a strong password. Use a password manager.
4. Save.

Hostinger writes a `.htaccess` (merged with yours) and a `.htpasswd` in a private location. Visiting the URL will now show a browser auth prompt.

Note

If hPanel overwrites or conflicts with the `.htaccess` from Step 3, open the file in File Manager afterwards and confirm the security-headers block is still present. If it has been removed, paste it back in below the auth directives.

7.2 Method B: Manual `.htpasswd` (if hPanel option absent)

If your plan doesn't expose the Password Protect feature, do it by hand.

First, generate the password file. From your laptop:

```
# Generate locally - replace yourname with whatever username you want
htpasswd -Bc inflam.htpasswd yourname
# (-B forces bcrypt; -c creates a new file)
# Enter password twice when prompted
cat inflam.htpasswd
```

Upload `inflam.htpasswd` to a location **outside** `public_html` — for example `/home/u12345/private/inflam.htpasswd`. (If your plan does not let you put files outside the web root, place it in `public_html` and rely on the `.htaccess` below to block direct access.)

Set its permission: `600`.

Then append to `inflam-7f3a92/.htaccess`:

```
# HTTP Basic auth
AuthType Basic
AuthName "Inflammation Tracker"
AuthUserFile "/home/u12345/private/inflam.htpasswd"
Require valid-user

# Defence in depth: refuse to serve the password file even if path leaks
<FilesMatch "\.htpasswd$">
  Require all denied
</FilesMatch>
```

Substitute your actual absolute path. After saving, reload the URL — you should now see a login prompt.

8 Step 5 — Create a Koofr app password

Never use your main Koofr password in the app's settings tab. Generate a dedicated, revocable app password instead.

1. Sign in to <https://app.koofr.net>.
2. Create a folder for the data, e.g. `/inflammation/`. Per-entry CSV files will be written here. (Optional — the app will create the folder on first submit if missing.)
3. Go to **Account** → **Preferences** → **Password** → **Application-specific passwords**.
4. Click **Add new** and give it a descriptive name (e.g. "inflam-pwa-phone").
5. Copy the generated password **immediately** — Koofr only shows it once.

Important

Koofr app-specific passwords are *account-wide*: they give the same access as your main password, just under a separately revocable credential. They cannot be limited to a single folder. If an app password leaks, treat it as if your whole Koofr were exposed: revoke it from the same page in Koofr settings, and the leaked credential becomes useless.

What you still get from using an app password rather than your main one:

- Independent revocation without disturbing your main login.
- Your main password (and its 2FA, if enabled) is never typed into this app or stored in any device's `localStorage`.

- You can issue one app password per device (phone, laptop) so a lost device only forces revocation of that single credential.

9 Step 6 — First configuration in the app

1. Open <https://yourdomain.example/inflam-7f3a92/> on your laptop. Enter the HTTP Basic auth credentials from Step 4.
2. Tap the gear icon (top right) to open settings.
3. Fill in:
 - **Koofr email:** the email on your Koofr account.
 - **Koofr app password:** the one generated in Step 5.
 - **Log path on Koofr:** `/inflammation/log.csv` (or whatever you prefer). This field is interpreted as a *template*: the directory part (`/inflammation/`) is where per-entry files are written, and the filename’s base (`log`, without `.csv`) becomes the prefix on each per-entry filename, giving paths of the form `/inflammation/log__<entry-date>__<write-ms>.csv`. The file at the configured path itself is not written to — a legacy single-file log can sit alongside the new files in the same folder.
 - **Proxy URL:** leave blank for now.
4. Click **Save settings**.
5. Click **Test connection**.

If Test connection succeeds

You’re done with setup. Go to Step 8.

If Test connection fails with “Network blocked... CORS”

This is expected for direct browser-to-Koofr requests in many cases. Proceed to Step 7 to deploy a proxy.

If Test connection fails with “Auth failed (401)”

Re-check the email and app password. Note Koofr may distinguish between your account email and a separate login name — use whatever you log in to `app.koofr.net` with.

10 Step 7 — Set up the Cloudflare Worker proxy

If Step 6’s **Test connection** failed with a CORS / “Network blocked” message, this step fixes it. Koofr’s WebDAV does not allow browsers to call it directly. You will deploy a tiny relay on Cloudflare that sits between your app and Koofr; it is free, takes about 5 minutes, and you only do it once.

You will need three things along the way. Open a notepad and keep these as you go:

- | | |
|---|--|
| (A) Your site’s HTTPS origin | e.g. <code>https://yourdomain.example</code> |
| (B) Worker URL | filled in at step 7.5 below |
| (C) Whether you already have a Cloudflare account | sign-up takes 1 minute |

Do these steps in order

7.1 Sign in to Cloudflare.

Go to <https://dash.cloudflare.com>. Create a free account if you don’t have one. You do not need a domain registered with Cloudflare for this.

7.2 Open the Workers section.

In the left sidebar click **Workers & Pages**. (If you don't see it, click the hamburger menu first.)

7.3 Create a new Worker.

Click **Create application** → **Create Worker**. Give it a name such as `koofr-proxy`. Click **Deploy** to accept the default placeholder. You will replace the code in the next step.

7.4 Edit the Worker code.

On the Worker's page click **Edit code** (top right). A code editor opens. Delete everything in it and paste this exactly:

```
// Replace the line below with your site's HTTPS origin.
const ALLOWED_ORIGIN = "https://yourdomain.example";

export default {
  async fetch(req) {
    if (req.method === "OPTIONS") {
      return new Response(null, { headers: corsHeaders() });
    }
    const url = new URL(req.url);
    const target = "https://app.koofr.net/dav/Koofr" + url.pathname;
    const init = {
      method: req.method,
      headers: req.headers,
      body: ["GET", "HEAD"].includes(req.method) ? undefined : req.body,
      redirect: "manual",
    };
    const upstream = await fetch(target, init);
    const headers = new Headers(upstream.headers);
    for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
    return new Response(upstream.body, {
      status: upstream.status,
      statusText: upstream.statusText,
      headers,
    });
  }
};

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": ALLOWED_ORIGIN,
    "Access-Control-Allow-Methods": "GET,PUT,POST,DELETE,PROPFIND,MKCOL,OPTIONS",
    "Access-Control-Allow-Headers": "Authorization,Content-Type,Depth,If-Match,If-None-Match",
    "Access-Control-Expose-Headers": "ETag",
    "Access-Control-Max-Age": "86400",
  };
}
```

Important

Change the `ALLOWED_ORIGIN` value (line 2) to your actual site URL — the one you put in (A) above. Use the exact scheme and domain, no trailing slash. If this doesn't match the site the request comes from, the browser will reject the response.

Click **Save and Deploy** (top right of the editor).

7.5 Copy the Worker's URL.

At the top of the Worker page, Cloudflare shows the public URL, looking like:

`https://koofr-proxy.<your-account>.workers.dev`

Copy it. Write it down as (B) above. No trailing slash.

7.6 Put the Worker URL into the app's settings.

Open your hosted app in the browser. Open the gear (Settings) tab. In the **Proxy URL** field paste the Worker URL from (B). Click **Save settings**, then **Test connection**. It should now report **Connection OK**.

7.7 Update your .htaccess so the browser is allowed to talk to the Worker.

This is the easy-to-miss step. Without it, your browser's Content Security Policy will still block the request.

Open the .htaccess file you created in Step 3 (using Hostinger File Manager → public_html/inflam-XXXX/). Find the line that begins:

```
Header always set Content-Security-Policy "...
```

Replace that one line with this (substituting your Worker URL):

```
Header always set Content-Security-Policy "default-src 'none'; script-src 'self'; style-src 'self'
'; img-src 'self'; manifest-src 'self'; connect-src 'self' https://app.koofr.net https://
koofr-proxy.<your-account>.workers.dev; worker-src 'self'; form-action 'none'; frame-
ancestors 'none'; base-uri 'none'"
```

Save the file.

7.8 Verify.

Reload the app in your browser (Ctrl/Cmd-Shift-R for a hard reload). Go to Settings, click **Test connection**. Then go back to the entry view and save a test entry. The CSV should appear in your Koofr at the path you configured.

Note

If you still get a CORS error after 7.7, the most common cause is a typo in either the Worker URL or the CSP line. Open browser DevTools (F12) → Console tab and read the exact CSP violation message — it will print which origin was blocked, which makes the mismatch obvious.

11 Step 8 — Install on your phone

11.1 Android (Chrome)

1. Open the URL in Chrome and enter the HTTP Basic auth credentials.
2. Chrome should offer “Install app” or “Add to home screen” via the menu (:).
3. Tap it; confirm. The icon appears on the home screen.
4. Launch it from the home screen. It runs full-screen without browser chrome.
5. Open settings inside the app and enter your Koofr email, app password, and (if used) Worker URL. These are stored in this device's browser storage only.

Note

Chrome on Android caches the HTTP Basic auth credentials for the session, but may prompt again after a long period. Save them in your phone's password manager.

11.2 Desktop (any Chromium browser)

In the address bar, an install icon appears once the manifest and service worker are detected. Click to install as a standalone window.

11.3 iOS (Safari)

Safari’s PWA support is more limited. “Add to Home Screen” from the share menu works for the icon, but offline support and the service worker may behave differently. Android Chrome is the recommended phone target.

12 Where your data is stored — a quick reference

Data	Stored at	Synced?
App source files (HTML/CSS/JS)	Your Hostinger account	—
Koofr email + app password	<code>localStorage</code> on each device, separately	No
Offline queue of unsynced entries	<code>localStorage</code> on each device, separately	No
Cached app shell (for offline use)	Service worker cache on each device	No
HTTP Basic auth credentials	Your password manager + browser keychain	No
Daily entries (the actual log)	Folder of one-row CSV files on Koofr, one file per entry	Yes

Only the entries folder on Koofr is shared across devices. Everything else is per-device.

If you clear browsing data for your hosting origin, the app’s settings and the unsynced queue on *that device* will be lost — but historical entries on Koofr are unaffected.

13 Maintenance and recovery

13.1 Rotating the Koofr app password

On Koofr settings, revoke the old app-specific password and generate a new one. Update the app’s settings tab on each device.

13.2 Resetting one device

Settings tab in the app, edit the fields, save. There is no other state to clear.

13.3 Losing a device

1. Sign in to Koofr from another device → revoke the app password used by the lost device.
2. Optionally change your Hostinger Basic auth password to invalidate the cached credential.

13.4 Backing up the entries folder

The folder of per-entry CSV files *is* your data. Koofr offers desktop sync clients that mirror it locally. Recommended: install a Koofr desktop client and let it sync the `/inflammation/` folder to a local directory included in your usual backups.

13.5 Inspecting the data

Each entry is its own CSV file consisting of a header row plus one data row, with a filename of the form `<prefix>__<entry-date>__<write-ms>.csv`, where `<prefix>` is the basename of the log path configured in Settings (e.g. `log` for `/inflammation/log.csv`). Including the header in every file is deliberate: each file is fully self-describing, so a file written under one schema version remains unambiguous to read even after later columns are added or removed. To work with the log as a whole, list the folder and load the files.

In Python with pandas, each file’s own header drives the parse:

```
import glob, pandas as pd
files = sorted(glob.glob("log_*.csv"))
df = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
```

For a single combined CSV in a shell, keep the first file’s header and skip headers on the rest:

```
awk 'FNR==1 && NR!=1 {next} {print}' $(ls -1 log_*.csv | sort) > combined.csv
```

The schema of each row is:

Column	Meaning
entry_id	UUID v4 generated at submit time. Stable across retries; uniquely identifies the entry independent of filename.
write_timestamp	ISO 8601 with local timezone offset (e.g. 2026-05-18T15:30:22+01:00). The moment the user tapped <i>Save entry</i> on this device, not the moment the file reached Koofr.
date	The entry’s <i>subject</i> date, YYYY-MM-DD. User-editable on the form for retrospective entries.
time	The entry’s <i>subject</i> time, HH:MM (24h, local). Likewise user-editable.
score	Inflammation severity, integer 1–4. 1 = no symptoms, 2 = minor but noticeable, 3 = moderate, 4 = severe.
locations	Pipe-delimited list of affected sites (e.g. <code>left wrist right knee</code>). Drawn from the ten fixed checkboxes <i>plus</i> any free-text value the user entered into the “Other:” field on the entry form. Empty string if no locations were ticked.
dietary_notes	Free-text dietary note (≤ 200 chars).
other_notes	Free-text general note (≤ 200 chars).
methotrexate	0 if not taken in the past 24 hours, 1 if taken.

Notes on parsing:

- Data fields are CSV-escaped (wrapped in double quotes; internal quotes doubled); the header line is plain unquoted column names. Excel, pandas and MATLAB handle the mix transparently.
- Per-entry files include a header row, so `pd.read_csv` / MATLAB’s `readtable` pick up the column names automatically with no extra arguments. If you concatenate at the shell level, skip duplicate headers (one `awk` approach is shown above).
- Locations are easiest to expand with `df["locations"].str.split("|").explode()` (pandas) or `strsplit(T.locations, '|')` (MATLAB).
- The `methotrexate` column was added partway through the project. Entries logged before that change will be missing the column and may parse as `NaN` — treat them as “unknown”, not “0”.
- `entry_id` and `write_timestamp` were added when the storage model switched from a single appended CSV to one file per entry. Rows imported from the older single-file log will be missing both — treat them as `NaN`.

13.6 Updating the app

Edit the source files locally and re-upload via SFTP or File Manager. The service worker uses a **network-first** strategy for same-origin shell assets (with cache fallback for offline use), so each PWA load fetches the latest `index.html` / `app.js` / `sync.js` from the server whenever the network is reachable and you are authenticated. A `CACHE_VERSION` bump in `sw.js` is therefore no longer mandatory for routine code changes; bump it only when (a) you change the `SHELL` array in `sw.js`, or (b) you want to force-evict every device’s cache from a known-good baseline.

14 Troubleshooting

The app loads but submit shows “Network blocked... CORS”

You need the Cloudflare Worker proxy (Section 7) and the corresponding CSP update.

Submit fails with “Auth failed (401)”

The Koofr email or app password is wrong, or the app password has been revoked. Regenerate and re-save.

Submit succeeds locally but no per-entry file appears on Koofr

Check the queue count in the top-right badge. Open settings → **Retry sync**. Read the error message there — it surfaces the underlying HTTP status.

App appears stuck on an older version after a deploy

Symptom: a feature you know was added in a recent deploy is missing on a device that has the PWA installed — e.g. per-entry files written without the CSV header even though that logic is in the deployed `sync.js`. Cause: the service worker’s background update attempt fetched a 401 from `.htaccess` because the device’s basic-auth session had expired. Older versions of `sw.js` used an `atomic cache.addAll(SHELL)` on install, so a single 401 aborted the entire SW update and silently stranded the device on the previously cached code; the runtime fetch handler was also cache-first, so the stale code kept running indefinitely.

The current `sw.js` hardens both halves: install uses individual `cache.add` calls under `Promise.allSettled` so partial failures are tolerated and logged via `console.warn`, and the fetch handler is network-first with cache fallback. A successful re-authentication via the foreground `.htaccess` prompt is enough for the next page load to pull fresh code from the server. If you ever see the symptom again on an unpatched device, the unblock is: fully close the PWA, open the site in a normal browser tab, complete the basic-auth prompt, then relaunch the PWA. As a stronger reset, “Clear site data” for the origin in browser DevTools nukes the SW and cache entirely (this also wipes `localStorage`, so sync any pending entries first).

Service worker not registering on the phone

Service workers require HTTPS. If your Hostinger SSL is not yet provisioned, wait a few minutes after first DNS pointing. Verify with `curl -I https://...` that the site responds 200 over HTTPS, then close and reopen the app.

Phone install option doesn’t appear

Confirm: HTTPS is working; `manifest.json` loads with content-type `application/manifest+json` (check in Chrome DevTools → Application → Manifest); `sw.js` registers without errors (DevTools → Application → Service Workers).

Headers absent from `curl -I` output

Hostinger plan may have restricted `mod_headers`. Open a support ticket — on shared plans this is unusual but possible. As a fallback, the same headers can often be set via hPanel’s Domain → Security section if exposed.

`.htaccess` causes 500 Internal Server Error

A directive is unsupported. Comment out blocks until you find the offender. Most commonly: `mod_headers` not loaded (rare on Hostinger) — you can wrap the `Header` lines in `<IfModule mod_headers.c>...</IfModule>`.

15 Security summary — what protects what

A compact mapping from threat to defence, so you can verify each is in place.

Threat	Defence in this setup
Someone discovers the URL and reads the app	HTTP Basic auth (Step 4); unguessable folder name (Step 1)
Network sniffing of credentials in transit	HTTPS enforced (Step 3); HSTS
Malicious script injection into the page	Strict Content Security Policy (Step 3); no inline JS/CSS in app code
Page embedded in an iframe for clickjacking	<code>X-Frame-Options: DENY + CSP frame-ancestors 'none'</code>
Leak of Koofr app password	Independently revocable from Koofr settings; one app password per device (Step 5)
Hostinger account compromise	Hostinger account 2FA (configure in hPanel); separation of credentials
Device loss / theft	OS disk encryption; revoke that device's Koofr app password
Browser malware reading <code>localStorage</code>	Mitigated by CSP (prevents injected script in app); not fully eliminated — consider passphrase-encryption add-on if at-risk
Koofr account compromise	Koofr account 2FA; app-specific passwords are independently revocable
Loss of unsynced queue	Sync regularly when online; CSV on Koofr is authoritative

Appendix — The `.htaccess` file (full)

For convenience, here is the complete recommended `.htaccess` combining all steps. Replace `<your-account>` with your Cloudflare account name (or remove the Worker entry if you don't use one), and adjust the `AuthUserFile` path.

```
# --- Force HTTPS ---
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [R=301,L]

# --- No directory listings ---
Options -Indexes

# --- Security headers ---
<IfModule mod_headers.c>
  Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains"
  Header always set X-Content-Type-Options "nosniff"
  Header always set X-Frame-Options "DENY"
  Header always set Referrer-Policy "no-referrer"
  Header always set Permissions-Policy "geolocation=(), microphone=(), camera=(), interest-cohort=()"
  Header always set Content-Security-Policy "default-src 'none'; script-src 'self'; style-src 'self'; img-src 'self'; manifest-src 'self'; connect-src 'self' https://app.koofr.net https://koofr-proxy.<your-account>.workers.dev; worker-src 'self'; form-action 'none'; frame-ancestors 'none'; base-uri 'none'"
</IfModule>

# --- MIME types ---
AddType application/manifest+json .json
AddType image/svg+xml .svg
AddType text/javascript .js

# --- HTTP Basic auth ---
AuthType Basic
AuthName "Inflammation Tracker"
```

```
AuthUserFile "/home/u12345/private/inflam.htpasswd"
Require valid-user

# --- Block direct access to the password file (defence in depth) ---
<FilesMatch "\.htpasswd$">
    Require all denied
</FilesMatch>
```

16 Your working setup — closing reference

What a fully working system looks like in your specific case. Use this as a memory aid when you’ve forgotten the details. Values in *red italics* are unknown to me at the time of writing — fill them in once and they’re recorded for future-you.

16.1 The pieces

Domain	<i><your-domain></i> (hosted at Hostinger)
App URL	<code>https://<your-domain>/<your-folder-name>/</code>
HTTP Basic auth	username <i><your username></i> , password in your password manager
Cloudflare account	<i><your-account></i>
Worker URL (proxy)	<code>https://koofr-proxy.<your-account>.workers.dev</code>
Worker dashboard	<code>https://dash.cloudflare.com</code> → Workers & Pages → koofr-proxy
Koofr account	login at <code>https://app.koofr.net</code>
Koofr log path	<code>/inflammation/log.csv</code> (folder <code>/inflammation/</code> , per-entry filename prefix <code>log</code>)
Koofr app password label	<i>e.g. “inflam-pwa-laptop”</i> (manage at Koofr → Account → Preferences → Password → Application-specific passwords)

16.2 What flows where

1. You open `https://<your-domain>/<your-folder-name>/` on phone or laptop. Hostinger serves the static files; Apache enforces HTTPS, the security headers, and the HTTP Basic auth challenge.
2. Once authenticated, the seven app files load. The service worker (`sw.js`) intercepts same-origin GETs and applies a network-first strategy with cache fallback: each load fetches fresh code from the server when reachable, falls back to the cached shell offline, and never overwrites the cache with a non-OK response (e.g. a 401 from an expired `.htaccess` session). The app runs entirely client-side.
3. **On-load check:** if Koofr credentials are saved in this browser’s `localStorage` and the device is online, the app issues a Depth-1 PROPFIND on the `entries` folder, counts filenames matching today’s prefix, and shows a non-blocking note (“Heads up: you already saved ...”) above the entry form if any exist. Submission is not blocked — duplicates are allowed.
4. You fill in the form and tap **Save entry**. The app generates a UUID (`entry_id`) and a local-timezone ISO 8601 `write_timestamp`, attaches both to the entry, pushes it to a local queue in `localStorage`, then PUTs to your Cloudflare Worker at `koofr-proxy.<your-account>.workers.dev`.
5. The Worker forwards the request to `app.koofr.net/dav/Koofr/inflammation/<prefix>__<date>__<ms>.c` with your Basic auth (email + Koofr app password) and adds CORS headers on the reply. The PUT carries an `If-None-Match: *` header so a retry that lands on an already-existing path is a no-op rather than an overwrite.

6. Koofr writes the per-entry CSV file. The Worker passes the response back; the app removes the entry from the queue.

16.3 Where each piece of state lives

App source (HTML/CSS/JS)	Hostinger public_html/ <i><your-folder-name>/</i>
.htaccess	Same folder. Contains HTTPS redirect, security headers, CSP (which must list the Worker URL), HTTP Basic auth directives.
.htpasswd	Outside public_html (e.g. /home/uXXXXX/private/inflam.htpasswd) or set up via hPanel's Password Protect feature.
Worker code	Cloudflare dashboard → koofr-proxy → Edit code. Critical line: <code>ALLOWED_ORIGIN = "https://<your-domain>"</code> .
Settings (per device)	<code>localStorage["settings"]</code> in each browser. Contains your Koofr email, app password, log path <code>/inflammation/log.csv</code> (used as a template for per-entry filenames), and proxy URL <code>https://koofr-proxy.<your-account>.workers.dev</code> .
Unsynced queue (per device)	<code>localStorage["queue"]</code> .
Cached app shell (per device)	Service worker cache <code>infram-shell-vN</code> .
The data	The per-entry CSV files in <code>/inflammation/</code> on Koofr, named <code>log__<date>__<write-ms>.csv</code> . This folder is the only authoritative store — everything else can be rebuilt from the source files.

16.4 Physical disk location of your credentials

The Koofr email + app password (and the offline queue) are written to the browser's `localStorage` under the JSON key `"settings"`. They never leave your devices except as Basic-auth headers sent to your Cloudflare Worker. The on-disk file format and location vary by browser:

On macOS

Browser	Path on disk (and notes)
Firefox	<code>~/Library/ApplicationSupport/Firefox/Profiles/<id>.default-release/storage/default/https+++<your-domain>/ls/data.sqlite</code> <i>SQLite database. Find your profile id by opening <code>about:profiles</code> in Firefox. Inspect via the Storage Inspector: right-click the page → Inspect → Storage tab → Local Storage.</i>
Chrome	<code>~/Library/ApplicationSupport/Google/Chrome/Default/LocalStorage/leveldb/</code> <i>Binary leveldb store. Replace Default with the profile directory name shown at <code>chrome://version</code> if you use multiple profiles.</i>
Edge	<code>~/Library/ApplicationSupport/MicrosoftEdge/Default/LocalStorage/leveldb/</code> <i>Same engine and format as Chrome.</i>
Safari	<code>~/Library/Containers/com.apple.Safari/Data/Library/WebKit/WebsiteData/LocalStorage/</code> <i>One SQLite file per origin. Web Inspector reads it via Develop → Show Web Inspector → Storage.</i>
Brave	<code>~/Library/ApplicationSupport/BraveSoftware/Brave-Browser/Default/LocalStorage/leveldb/</code> <i>Chromium-based, identical layout to Chrome.</i>

On Android

Browser	Path on disk (and notes)
Chrome (<i>also the home-screen PWA</i>)	/data/data/com.android.chrome/app_chrome/ Default/LocalStorage/leveldb/ <i>Inside Chrome's private app sandbox. Other apps cannot read it; the file manager will not show it unless the device is rooted. An installed PWA shares this store — it does not get its own.</i>
Edge	/data/data/com.microsoft.emmx/app_chrome/ Default/LocalStorage/leveldb/ <i>Same Chromium pattern.</i>
Samsung Internet	/data/data/com.sec.android.app.sbrowser/ app_sbrowser/Default/LocalStorage/leveldb/ <i>Chromium-based.</i>
Firefox for Android	/data/data/org.mozilla.firefox/files/ mozilla/<profile>/storage/default/<origin> /ls/data.sqlite <i>SQLite, mirroring desktop Firefox.</i>

Note

On Android, all paths above are inside per-app private storage. You cannot reach them with a normal file manager. Use remote DevTools (below) to inspect.

On an iPhone or iPad — iOS / iPadOS

Browser	Path on disk (and notes)
Safari	<code>/var/mobile/Containers/Data/Application/<UUID>/Library/WebKit/WebsiteData/LocalStorage/</code> <i>SQLite per origin. Path is not user-accessible without jailbreaking. To clear: Settings → Safari → Advanced → Website Data → find the site.</i>
Chrome / Edge / Firefox / any other iOS browser	Same WebKit storage subsystem inside each app's own container. <i>Apple requires every browser on iOS to use WebKit; storage mechanics are identical but each app's container is isolated, so credentials entered in Chrome on iOS are not visible to Safari on the same phone.</i>
Installed PWA (“Add to Home Screen”)	Each home-screen webapp gets a separate WebKit container. <i>Since iOS 17, this storage is increasingly isolated from regular Safari. Treat the PWA and Safari as two different stores: credentials entered in one will not appear in the other.</i>

Inspecting or clearing the stored values

The on-disk files are binary or SQLite — do not try to grep them directly. Use the browser's developer tools instead:

- **Mac, Firefox:** right-click the page → **Inspect** → **Storage** tab → **Local Storage** → your origin → the `settings` key shows the decoded JSON.
- **Mac, Chrome / Edge / Brave:** F12 → **Application** tab → **Local Storage** → your origin.
- **Mac, Safari:** enable *Develop* menu in Preferences → Advanced → “Show Develop menu”, then **Develop** → **Show Web Inspector** → **Storage**.
- **Android phone:** enable USB debugging in Android Developer Options, connect the phone to your Mac, open `chrome://inspect/#devices` in desktop Chrome, find the app under the phone's entry, click **inspect**. DevTools then opens for the remote page.
- **iPhone / iPad:** enable Web Inspector in iOS Settings → Safari → Advanced; connect the device to your Mac by Lightning/USB-C cable; on the Mac open Safari → Develop menu → select your iPhone → the page.
- **To wipe credentials on any device:** either edit the Settings tab in the app and clear the fields, or use the browser's “Clear site data” for the origin. Both erase the offline queue too, so sync anything pending first.

Note

The values on disk are not encrypted by the browser; they are protected by OS-level mechanisms only (your macOS user permissions and FileVault, Android’s app sandbox and device encryption, iOS’s app sandbox). Anyone with an unlocked, signed-in session on the device can read them via the steps above. This is why the manual recommends one Koofr app password per device — losing a device only forces revocation of that single credential.

16.5 How to deploy a change

1. Edit the relevant file(s) locally.
2. Upload the changed files to Hostinger (File Manager or SFTP), preserving the folder structure.
3. If you changed the CSP or the Worker URL, update `.htaccess` accordingly and re-upload.
4. If you added or removed files in the `SHELL` array in `sw.js`, bump `CACHE_VERSION` so the new shell layout takes effect cleanly. For routine code changes (editing `app.js`, `sync.js`, `style.css`, `index.html`) the network-first fetch handler picks up the new file on the next page load, so no bump is required.
5. Hard-reload the app on your laptop (Cmd-Shift-R) and confirm. On your phone, fully close the PWA and reopen it. The new code should be live; if it is not, see the *App appears stuck on an older version after a deploy* entry under Troubleshooting.

16.6 Sanity-check routines

If something stops working, run through this in order:

1. Does `https://<your-domain>/<your-folder-name>/` load and prompt for HTTP Basic auth? If not, check Hostinger SSL status and the `.htaccess`.
2. Settings tab → **Test connection**. Failure messages map as follows:
 - “Auth failed (401)” — Koofr email or app password wrong. Regenerate at Koofr if in doubt.
 - “Network blocked / CORS” — The Worker URL in Settings is wrong, the Worker is down, or the CSP in `.htaccess` doesn’t include the Worker URL. Open browser DevTools → Console for the exact CSP violation.
3. Browser DevTools → Network: every successful PUT to the Worker should return 201 (created), since each save writes a new per-entry file. A 412 (Precondition Failed) coming back is not an error — it means a retry hit a file that an earlier attempt had already written, which the app treats as success.
4. Worker still alive? Cloudflare dashboard → Workers & Pages → `koofr-proxy`. The Worker is free-tier and effectively never expires, but if you ever delete it accidentally, re-create from the code template in Step 7.

16.7 Backup

The per-entry CSV files in `/inflammation/` on Koofr are the only place that contains your data. Install the Koofr desktop client (from `https://koofr.eu`) on your Mac, point it at `/inflammation/`, and let it mirror to a local folder included in Time Machine. Then a Koofr outage, an accidental delete, and a laptop loss all need to happen together for the data to be unrecoverable.

16.8 If you ever need to rebuild from scratch

You’d need: the seven app files, this manual, the Cloudflare Worker code (Step 7), and these credentials — Hostinger login, Cloudflare login, Koofr login, and the Basic auth password. With those, the whole system can be reconstructed end-to-end in about an hour.

End of manual. Iterate as your setup evolves — the security baseline above is the starting point, not the ceiling.